

Índice

1. Seguridad en la Empresa	2
1.1. Gobernanza (Governance)	2
1.2. Gestión de riesgos (Risk Management)	2
1.2.1. Identificación del riesgo	3
1.2.2. Evaluación del riesgo	3
1.2.3. Mitigación del riesgo — las 5 opciones	3
1.2.4. Monitoreo	3
1.3. Políticas de seguridad	4
1.4. Estrategias: Defense-in-depth vs. Zero Trust	4
1.4.1. Defense-in-depth (defensa en profundidad)	4
1.4.2. Zero Trust Architecture (ZTA)	7
1.5. Privileged Access Management (PAM)	8
1.6. Prevención de vulnerabilidades: DevSecOps	8
1.7. Protección de la nube	10
1.8. Evaluaciones de seguridad	10
1.8.1. Penetration Test (PenTest)	11
1.8.2. Red / Blue / Purple Team	11
1.8.3. Breach and Attack Simulation (BAS)	12

1. Seguridad en la Empresa

Esta unidad cambia el foco: deja la matemática y las vulnerabilidades de aplicación y mira cómo una **organización real** (corporación o PyME avanzada) se protege. El profe aclaró que **este tema no está en Bishop** (salvo lo que se pueda leer suelto); la clase está muy enfocada en el panorama profesional contemporáneo y en el estándar **NIST 800-207** (Zero Trust).

“Esto de clase que vamos a ver hoy no está en Bishop. Está más enfocado en el panorama contemporáneo, lo profesional, cómo se maneja esto en una organización.”

— dicho en clase

► En el parcial

La lectura recomendada de esta clase es **NIST 800-207 (capítulos 2 y 3)**, no Bishop. Toda la parte de Zero Trust sigue ese estándar.

1.1. Gobernanza (Governance)

Definición: Gobernanza de seguridad

Conjunto de **procesos, políticas, procedimientos y controles** que se implementan para gestionar y mitigar los riesgos de la organización, y el establecimiento de **responsabilidades y roles** para garantizar la protección de la información. Es como el “gobierno” interno: emite las reglas y se hace responsable de que se cumplan.

La gobernanza la lleva (generalmente) el equipo de seguridad. Su propósito es asegurar **confidencialidad, integridad y disponibilidad** de los datos, tanto de forma **proactiva** (poner barreras y controles) como **reactiva** (qué medidas tomo una vez atacado).

Objetivos principales:

- **Alinear la estrategia de seguridad con los objetivos del negocio.** No poner medidas que vayan en contra de cómo la empresa gana dinero.
- Reducir y mitigar los riesgos de la operación.
- Cumplir regulaciones y normativas aplicables (p.ej. empresas que cotizan en bolsa deben informar incidentes; datos personales mal protegidos ⇒ multas millonarias).
- **Crear cultura de seguridad** (campanas de concientización, de phishing, capacitación), porque el eslabón más débil es el usuario.

! Importante — remarcado en clase

El eslabón más débil es siempre el **usuario / empleado / cliente**. Por eso, si se ponen **demasiadas trabas**, el empleado busca atajos (se baja los datos a un CSV, los sube a Drive, etc.) y **termina atentando contra la seguridad**. Esto es el problema de la *aceptación psicológica*: la estrategia debe estar alineada con el negocio para no friccionar el trabajo.

1.2. Gestión de riesgos (Risk Management)

Históricamente la seguridad se gestionaba “como un riesgo más”, y ese modelo se mantiene. Es un **ciclo que nunca para** (los riesgos evolucionan, aparecen nuevos): **Identificar** → **Evaluar** (Assess) → **Mitigar** → **Monitorear** y vuelta a empezar.

1.2.1. Identificación del riesgo

Primero se identifican los **activos críticos**, que involucran: **Datos** (de empleados, clientes, comerciales, financieros, proveedores), **Sistemas, Infraestructura** (servidores, oficinas, data centers) y **Personas**.

Luego los **riesgos potenciales**: Malware, Ataques externos, Errores humanos, Sabotaje (interno) y Ambientales (huracán, incendio). El profe sumó los **problemas externos** (caída de una región de un proveedor cloud que voltea a muchas empresas a la vez).

1.2.2. Evaluación del riesgo

Se estima **probabilidad de ocurrencia** $\times$ **impacto en el negocio** (matriz probabilidad vs. impacto). Se clasifican en **Alto** / **Medio** / **Bajo** y se atacan primero los de mayor probabilidad y mayor impacto. Acá también se realizan evaluaciones externas (Red Team / PenTest).

1.2.3. Mitigación del riesgo — las 5 opciones

Definición: Tratamiento de un riesgo

Ante un riesgo identificado hay cinco maneras de manejarlo:

1. **Evitarlo**: eliminar la causa. Suele ser *costoso/complejo* (p.ej. resolver vulnerabilidades de un sistema legacy o migrarlo).
2. **Reducirlo** (lo más común): bajar el impacto sin eliminarlo (p.ej. sumar un factor extra de autenticación a una autenticación débil).
3. **Transferirlo**: delegarlo a un proveedor (p.ej. Cloudflare como proxy reverso). Si pasa algo, responde el proveedor.
4. **Compensarlo**: no se puede reducir, pero se contrarresta el impacto (p.ej. auditar/controlar continuamente una dependencia que no se puede quitar).
5. **Aceptarlo**: se entiende y se asume el riesgo (típico en riesgos de baja probabilidad). **Se debe documentar** la decisión.

► En el parcial

Distinguir **reducir** (baja la probabilidad/impacto del riesgo) de **compensar** (el riesgo sigue igual, pero se contrarresta su efecto). Un ejemplo de **aceptar**: muchas empresas tienen un **DRP** (Disaster Recovery Plan) documentado y exigido (p.ej. el Banco Central a las fintech), pero **no lo ejecutan** por costo $\Rightarrow$ aceptan el riesgo y lo dejan documentado.

1.2.4. Monitoreo

Es cíclico y permanente (la organización es un sistema vivo). Existen **frameworks** de apoyo: NIST (análogo del IRAM en Argentina) y los **CIS Controls** (18 controles) como guía de qué tener en cuenta.

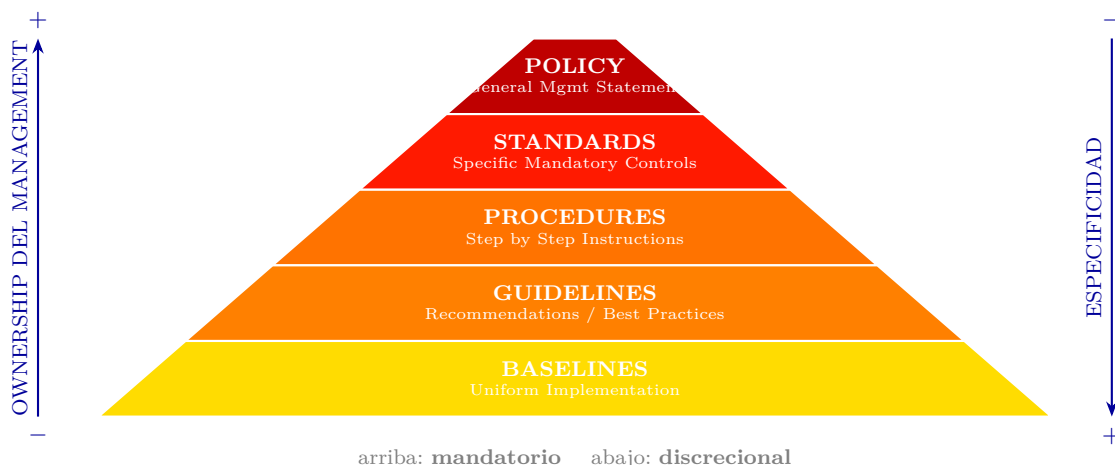
1.3. Políticas de seguridad

Definición: Política de seguridad

Documento **genérico y muy abarcativo** (“la Biblia” de la seguridad de la empresa); se confecciona cada cierto tiempo y lo aprueba la dirección (CISO/CEO). Define todo lo que compete a la seguridad, llegando incluso al plano legal (manejo de datos personales, estándares de industria).

De la política se va “bajando línea” con creciente especificidad (y decreciente *ownership* del management). Pirámide:

- **Policy** (declaración general del management) — obligatorio.
- **Standards** (controles específicos mandatorios).
- **Procedures** (instrucciones paso a paso).
- **Guidelines** (recomendaciones / mejores prácticas).
- **Baselines** (formas uniformes de implementación; cursos, capacitaciones, concientización).



Pirámide de políticas: hacia arriba crece el ownership del management y la obligatoriedad; hacia abajo crece la especificidad.

△ Cuidado — error típico

En la pirámide, hacia **arriba** crece el **ownership del management** y es **mandatorio**; hacia **abajo** crece la **especificidad** (y se vuelve más discrecional). No confundir el eje.

1.4. Estrategias: Defense-in-depth vs. Zero Trust

Surgieron estrategias genéricas para implementar las políticas. La clase destaca dos: **Defense-in-depth** (defensa en profundidad) y **Zero Trust Architecture (ZTA)**.

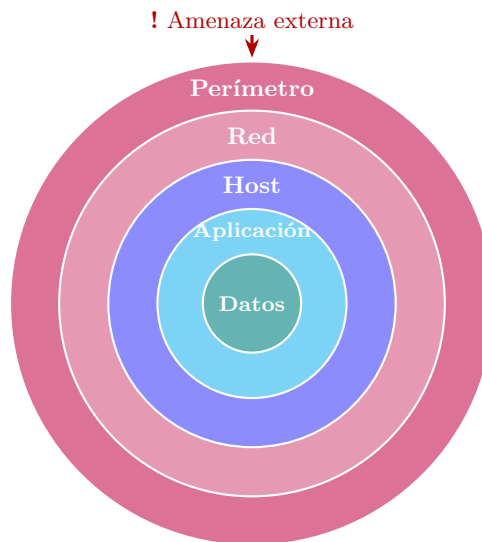
1.4.1. Defense-in-depth (defensa en profundidad)

Asume que **ningún control es perfecto**. Diseña **capas**, cada una pensada para contener a la siguiente (modelo “cebolla”), dando **redundancia**: si falla una capa, la de abajo contiene.

Las **5 capas** (de afuera hacia el dato):

1. **Perímetro (Perimeter)**: frontera con Internet. Firewall que deniega lo que viene de afuera y solo acepta cierta red o VPNs; honeypot; seguridad de mensajería (anti-virus/anti-malware); IDS/IPS de perímetro.
2. **Red (Network)**: dentro de la red corporativa. **Segmentación** (separar la red de operaciones de la red de servidores); NAC; Web Proxy / Content Filtering; DLP.
3. **Host / Endpoint**: *hardening* (“jardinizar”) del host — dejarlo inmutable, solo con el software permitido (idea tipo contenedor); Host IDS/IPS; antivirus.
4. **Aplicación**: buen diseño y buenas prácticas para no meter vulnerabilidades; **WAF**; monitoreo/escaneo de BD.
5. **Datos**: lo que más importa. Clasificación, cifrado, gestión de identidades y accesos (IAM), DLP, PKI.

Para llegar al dato, el atacante debe pasar perímetro → moverse lateralmente a la red de servidores (que no debería ser alcanzable) → vulnerar el host jardinizado → vulnerar la aplicación.



Defense-in-depth (modelo “cebolla”): cada capa contiene a la siguiente. El atacante debe atravesar Perímetro → Red → Host → Aplicación para llegar al Dato.

Principios.

- **Diversificación**: distintas tecnologías/métodos ⇒ una sola vulnerabilidad no atraviesa todo.
- **Redundancia**: cada capa contiene a la siguiente.
- **Segmentación**: dividir la red en segmentos controlados (se puede llegar al extremo de que una aplicación solo “vea” a la aplicación con la que necesita hablar).
- **Monitoreo**: visibilidad de todas las capas y **correlación de eventos** (si no se monitorea, nada de lo anterior importa).

Problemas.

- **Altos costos**: de tecnología (muchos proveedores), de capacitación/operación y de monitoreo (se recolectan muchos datos pero se les saca poco valor).
- **No resuelve bien los ataques internos** (si alguien ya está adentro, es difícil detectarlo; muchas herramientas no son bidireccionales).

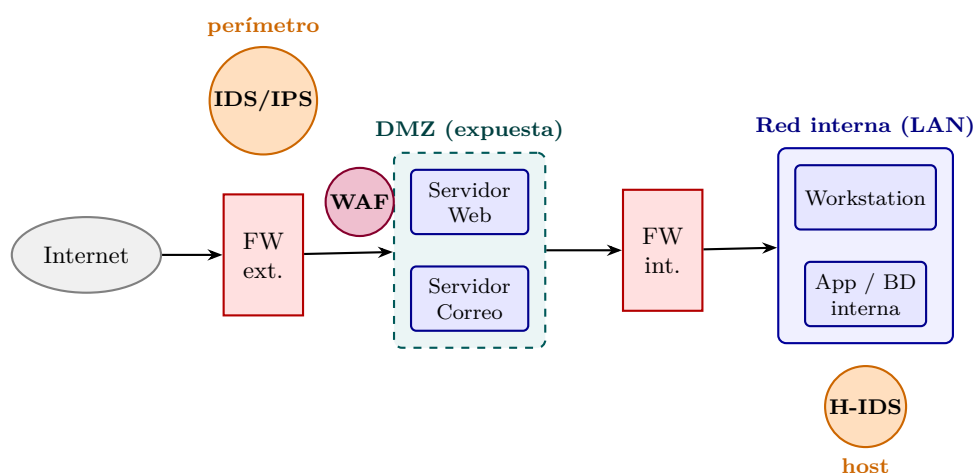
- **Difícil en infraestructura distribuida:** pensado para *on-premise*/data center propio; hoy hay data centers híbridos, multi-nube y empresas *cloud native*.

△ Cuidado — error típico

Firewalls en esta unidad (según lo visto en clase): el firewall de **perímetro** bloquea/filtra tráfico de red; el **WAF (Web Application Firewall)** aparece en la capa de **aplicación** y es “un firewall pero para el tráfico de una aplicación web”. En clase se mencionaron solo estos dos (firewall de perímetro y WAF).

! Importante — remarcado en clase

DMZ (zona desmilitarizada). En el diagrama de Defense-in-depth aparecen las *Secure DMZs* dentro de la seguridad de **perímetro/red**: es la zona expuesta hacia la red externa.



Topología con DMZ. El **firewall externo** expone solo la DMZ (web, correo); el **firewall interno** protege la LAN. El **IDS/IPS** se ubica en el perímetro y también en el host (Host IDS); el **WAF** se coloca delante de la aplicación web.

IDS / IPS.

Definición: IDS / IPS

Sistemas de **detección** (IDS, *Intrusion Detection System*) y de **prevención** (IPS, *Intrusion Prevention System*) de intrusiones. Funcionan “tipo antivirus”: detectan patrones a través de **firmas** o **anomalías** y determinan si **bloquear** (prevenir), **alertar** o tomar alguna acción.

► En el parcial

La diferencia clave: **IDS detecta** (alerta), **IPS previene** (bloquea). En el diagrama de Defense-in-depth aparecen tanto en el **perímetro** como en el **host** (Host IDS/IPS).

Otros componentes vistos en las capas.

- **Honeypot:** red “boba” a donde se deriva al atacante detectado en el perímetro para mantenerlo entretenido y que no llegue a la red que importa.
- **NAC (Network Access Control):** define si un dispositivo se puede conectar o no (por IP, MAC, firmware, credenciales); similar a limitar por MAC en un router.

- **Web Proxy / Content Filtering** (p.ej. Fortinet): bloquea/filtra tráfico (que la gente no entre a sitios maliciosos o no laborales).
- **DLP (Data Loss Prevention)**: evita la **exfiltración** de datos hacia afuera (escanea por datos de tarjeta, personales, etc.).

1.4.2. Zero Trust Architecture (ZTA)

Definición: Zero Trust Architecture (NIST 800-207)

Modelo de seguridad que asume que las amenazas **internas y externas están presentes en la red en todo momento**. Objetivo: proteger integridad, confidencialidad y disponibilidad de datos y recursos. Principios: **Nunca confíes (Never Trust)**, **Siempre verifica (Always Verify)**, **Asume que has sido atacado (Assume Breach)**.

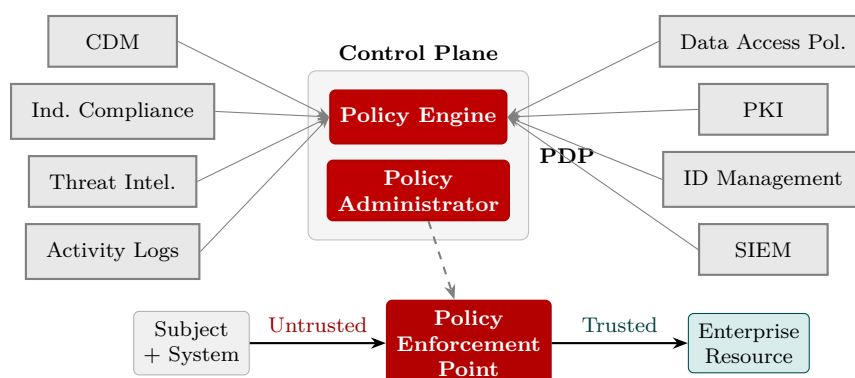
ZTA nace de constatar que **ninguna red es segura**: la workstation de un empleado (en la oficina o por VPN) puede tener malware sin que lo sepa, o puede haber un intruso físico. Por eso **cualquier red interna pasa a considerarse tan peligrosa como Internet**, y los controles que se aplicaban “afuera” ahora se aplican también “adentro”.

! Importante — remarcado en clase

La **intranet ya no es de confianza**. Antes era “buena práctica” relajar controles dentro de la red corporativa (para no friccionar al empleado); ZTA revierte eso: **cada componente/aplicación debe forzar la verificación**. En la práctica se relaja con **federación de identidad / SSO** (el empleado loguea una vez, p.ej. contra el directorio, y cada aplicación hereda su identidad, roles y permisos sin pedir login nuevo, pero *sí* aplicando los controles).

Componentes (NIST 800-207). El núcleo es el **Policy Decision Point (PDP) = Policy Engine + Policy Administrator**, y el **Policy Enforcement Point (PEP)** que separa la zona *Untrusted* (Subject/System) de la *Trusted* (Enterprise Resource). Lo alimentan:

- **CDM (Continuous Diagnostics and Mitigation)**: informa al motor de políticas sobre el activo que pide acceso (¿antivirus actualizado?, ¿últimos updates de seguridad?); puede desactivar el recurso si no cumple.
- **Industry Compliance**: garantiza el cumplimiento de regímenes normativos — **HIPAA, Banco Central, SOX, PCI**.
- **Threat Intelligence**: fuentes internas/externas sobre CVEs, firmas de ataques, IPs comprometidas, botnets (p.ej. VirusTotal).
- **Activity Logs**: registros de actividad de red y sistemas (datos crudos, en tiempo casi real).
- **SIEM (Security Information and Event Management)**: recolecta y **correlaciona** eventos para detectar anomalías y generar alertas.
- **ID Management**: crea/almacena/gestiona identidades de los usuarios (una persona puede tener varios usuarios), con sus roles y permisos.
- **PKI**: genera y registra los certificados de sujetos, recursos, servicios y aplicaciones.
- **Data Access Policy**: reglas/políticas de acceso a los recursos; punto de partida para autorizar (codificadas o generadas dinámicamente por el PE).



*Zero Trust (NIST 800-207): el **PDP** (Policy Engine + Policy Administrator) decide; el **PEP** aplica y separa la zona Untrusted (Subject) de la Trusted (Enterprise Resource), con verificación continua alimentada por CDM, Compliance, Threat Intel, Logs, PKI, ID Mgmt, SIEM y Data Access Policy.*

△ Cuidado — error típico

ZTA “parece la panacea” pero es **muy fácil de implementar mal**: o se implementa de forma deficiente, o se implementa y **no se controla / no se hace evolucionar**. Tener ZTA “puesto” no protege si no se monitorea y audita. Si una empresa arranca **de cero**, conviene ir directo a ZTA (sin el sesgo/lastre del esquema viejo).

1.5. Privileged Access Management (PAM)

Definición: PAM

Práctica para **proteger y gestionar cuentas privilegiadas** (acceso a sistemas críticos). Estos accesos son sensibles porque pueden conceder capacidades amplias, modificar configuraciones de seguridad, acceder a datos confidenciales y supervisar otras cuentas. Son lo que más buscan los atacantes.

Prácticas/herramientas:

1. **Inventario** de cuentas privilegiadas (visibilidad y control).
2. **Privilegio mínimo**: la cuenta privilegiada **no** es la cuenta de trabajo de la persona; es una cuenta separada (que puede transferirse al cambiar de rol).
3. **MFA** (autenticación multifactor) para garantizar acceso solo de autorizados y el **no repudio**.
4. **Registro y monitoreo** de toda la actividad (logs en servidor externo para reconstruir qué pasó).
5. **Rotación de contraseñas** (regular y automática; y reválida al transferir la cuenta).

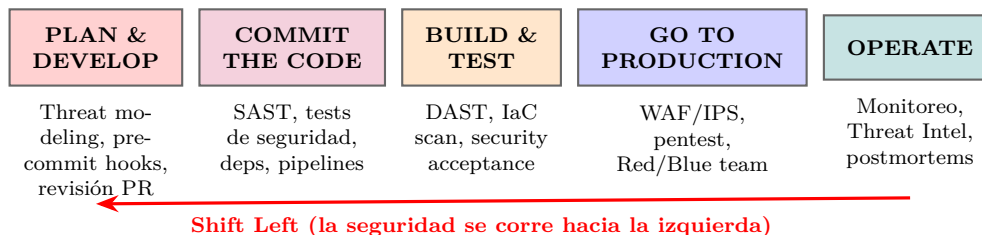
Ciclo de vida PAM: Define → Discover → Manage & Protect → Monitor → Detect Abnormal Usage → Respond to Incidents → Review & Audit.

1.6. Prevención de vulnerabilidades: DevSecOps

Definición: DevSecOps

Metodología que mete la **seguridad en cada paso** del proceso de llevar algo a producción, integrándola dentro de los equipos de desarrollo. Es la evolución de **DevOps** (que unió Dev y Ops bajo un mismo objetivo: “*you build it, you run it*”) sumándole el equipo de seguridad.

El problema histórico: Dev quiere sacar features rápido; Seguridad quiere frenar todo para revisar. Hoy la mayoría aplica seguridad recién en la **última etapa** (operación). DevSecOps propone el **Shift Left**: meter la seguridad lo más a la izquierda posible (planificación/desarrollo).



Pipeline DevSecOps: la seguridad se integra en cada etapa (plan/develop → commit → build/test → production → operate). El Shift Left empuja los controles lo más temprano posible.

“Cuanto más temprano detectamos un problema, menos difícil es resolverlo.”

— dicho en clase

Seguridad por etapa (Shift Left):

- **Plan & Develop: Threat modeling** (evaluar amenazas de una solución nueva antes de desarrollarla), plugins de seguridad en el IDE, **pre-commit hooks** (que no se expongan secretos), buenas prácticas, revisión de PRs.
- **Commit the code / PR: SAST** (análisis estático, p.ej. SonarQube), unit/functional tests de seguridad (foco en **CWE**), control de vulnerabilidades en dependencias, **pipelines seguros** (protección de secretos).
- **Build & Test: DAST** (análisis dinámico en sandbox), validación de configuración cloud / IaC, infrastructure scanning, security acceptance testing.
- **Go to Production:** evaluación de la app como un todo incluyendo el ambiente de despliegue (Firewalls, WAF, IPS), controles sobre artefactos y secretos, **pentests externos, Red/Blue team**.
- **Operate:** monitoreo continuo de métricas y logs, integración con Threat Intelligence, blind pentest, simulación de incidentes, respuesta a incidentes, *blameless postmortems*.

Testeos de seguridad en el código.

- **SAST** (Static): analiza el código *sin ejecutarlo*; marca malas prácticas, code smells y problemas de seguridad.
- **DAST** (Dynamic): evalúa la app **en ejecución**, interactuando como un usuario externo a través de las interfaces estándar (APIs, endpoints); busca fallas explotables en red (validación de entrada, autenticación). Automatiza pruebas repetibles (escanear puertos, inputs inválidos, etc.).
- **SCA** (Software Composition Analysis): analiza **dependencias y librerías** para identificar vulnerabilidades (incluso **transitivas**), problemas de licencia y otros riesgos. Debe extenderse a servicios SaaS consumidos.

! Importante — remarcado en clase

Caso real mencionado: **Log4Shell** (Log4j, 2021). Una vulnerabilidad crítica de *ejecución remota de código* en una librería de logging de Java usadísima. El problema: muchas empresas **no sabían si la usaban** (era una dependencia tan estándar que “te olvidás que está”). De ahí el resurgir del **SBOM**.

Definición: SBOM (Software Bill of Materials)

Inventario **formal y legible por máquina** de los componentes de software, dependencias, información sobre ellos y sus relaciones. Debe ser lo más completo posible (e indicar dónde no se pueden articular relaciones) e incluye software open-source y comercial. Permite saber qué componentes usa cada aplicación ante un incidente.

1.7. Protección de la nube

La configuración de la nube es **responsabilidad del cliente**. La **Infraestructura como Código (IaC)**, p.ej. Terraform, permite definir la configuración de forma reproducible; existen análisis estáticos que detectan configuraciones inseguras o ineficientes (puertos expuestos, redes abiertas).

Herramientas (productos caros, pensados para empresas con miles de servidores):

- **CSPM (Cloud Security Posture Management)**: busca problemas de configuración y permisos; analiza IaC *antes* de desplegar y revisa la config actual. Enfoque más **estático**.
- **CWPP (Cloud Workload Protection Platform)**: protege las aplicaciones corriendo en la nube (VMs, contenedores, serverless); analiza **comportamiento**, microsegmentación a nivel de contenedores, threat intelligence. Complementa a CSPM.
- **CIEM (Cloud Infrastructure Entitlement Management)**: foco en los **permisos** de los sujetos sobre recursos cloud; busca permisos excesivos comparando uso real vs. permisos otorgados; visualiza permisos entre cuentas.
- **CNAPP (Cloud Native Application Protection Platform)**: plataforma que integra lo anterior (visión seguridad proactiva + reactiva).

1.8. Evaluaciones de seguridad

“Tomamos todos los recaudos, todas las mejores prácticas... y en algún momento ese sistema va a fallar. No pregunten cuándo, no pregunten cómo, pero va a fallar.”

— dicho en clase

La seguridad nunca está asegurada $\Rightarrow$ hay que evaluar **constantemente**. Tipos de evaluación:

- **Evaluación de vulnerabilidades**: herramientas **automatizadas** que escanean en busca de vulnerabilidades **conocidas**.
- **Pentesting**: más exhaustivo, lo hacen **expertos** que intentan explotar vulnerabilidades de forma controlada.
- **Evaluación de riesgos**: identificar/evaluar/priorizar riesgos e impacto (la seguridad como una amenaza más).
- **Auditoría de seguridad**: revisión sistemática de políticas, procedimientos y controles (interna o externa).

- **Evaluación de cumplimiento:** obligatoria para regulaciones (**GDPR, HIPAA, PCI-DSS**, protección de datos personales).
- **Seguridad física:** control de acceso a instalaciones, cámaras, protección de archivos físicos y equipos.
- **Seguridad de aplicaciones:** revisión de código, pruebas de seguridad y análisis de arquitectura.

1.8.1. Penetration Test (PenTest)

Objetivo: agarrar el sistema **funcionando** y probarlo integralmente para identificar las vulnerabilidades que lo hacen pasar de un estado **seguro** a uno **inseguro**. Se hace con **alcance y tiempo definidos** (no se puede probar eternamente).

Tipos según visibilidad del evaluador:

- **Black Box / Blind:** el evaluador no tiene información previa (quizás solo el dominio/IP). Simula un atacante externo; favorece pensar “fuera de la caja” y descubrir vectores que el sesgo ocultaría. Razón práctica: no querer compartir info interna (motivos legales).
- **White Box:** acceso completo (código fuente, diagramas, credenciales). Evaluación exhaustiva de vulnerabilidades internas.
- **Gray Box:** intermedio (algún conocimiento, no todo). Simula un usuario con acceso limitado (empleado con privilegios restringidos).
- **Double Blind:** Black Box donde el **atacado tampoco sabe** del ejercicio; evalúa además las capacidades de monitoreo y respuesta. Es lo más parecido a la realidad (relacionado con Chaos Monkey).

Tipos según sistema evaluado: Red externa (servicios expuestos a Internet), Red interna (atacante interno: empleado descontento o malware en una workstation), Redes inalámbricas (NFC, Bluetooth, Wireless), Aplicaciones web (SQLi, XSS, etc.) e **Ingeniería social** (phishing, smishing).

Etapas (6 pasos): 1) Preparación (objetivo, alcance, tiempo, nivel de riesgo aceptado) → 2) Armado del plan de ataque (herramientas, ubicación) → 3) Identificar el equipo → 4) Definir el objetivo a alcanzar → 5) Ejecutar el ataque (termina al lograr el objetivo o al vencerse el tiempo) → 6) Integrar los hallazgos y reiniciar el ciclo (pivotar/escalar con lo obtenido).

1.8.2. Red / Blue / Purple Team

- **Red Team** (ofensivo): piensa y actúa como atacante malintencionado de forma controlada y ética — pentesting, ingeniería social, assessment de vulnerabilidades.
- **Blue Team** (defensivo): implementa y mantiene controles (firewalls, IDS, gestión de parches y configuraciones), monitorea y responde a incidentes.
- **Purple Team:** enfoque colaborativo que combina ambos para mejorar la postura de seguridad (fomenta comunicación en vez de aislamiento).

1.8.3. Breach and Attack Simulation (BAS)

Categoría de herramientas/técnicas que evalúan la efectividad de las defensas simulando las **tácticas, técnicas y procedimientos (TTP)** de un atacante real. Objetivo: identificar vulnerabilidades **y** medir la capacidad de la organización para **detectar y reaccionar**. Se ejecutan de forma **recurrente** en distintos puntos. Relacionado: *Breach Attack Simulation*/juegos de guerra (simular un escenario desfavorable puntual — “¿qué pasa si un atacante obtiene una ruta de admin del cloud?” — y analizar qué tácticas de defensa aplicar); es barato (juntar gente y plantear el escenario, a veces en ambiente controlado).